

Department of Statistics & Computer Science
University of Kelaniya
Academic Year -2024/2025 (Semester II)

COSC 32133/BECS 32263

Full-Stack Software Development

Assignment 02

Secure RESTful API for Hospital Management System using Spring Boot, Spring Security 6, JWT Authentication & Postman

1. Learning Outcomes

By the end of this lab, you will be able to:

- Implement stateless authentication using JSON Web Tokens (JWT) in Spring Boot 3 / Spring Security 6.
- Design Role-Based Access Control (RBAC) for system roles (ADMIN, DOCTOR, PATIENT).
- Secure REST API endpoints using @PreAuthorize and SecurityFilterChain.
- Encrypt sensitive data such as user passwords using BCryptPasswordEncoder.
- Perform API testing for protected endpoints (Auth, Doctors, Patients, Appointments) using Postman.
- Inspect generated tokens, verify payloads, and test bearer token authentication headers.

2. Scenario

A healthcare organization requires a RESTful Backend API for a **Small Hospital Management System**. Access control must be strictly enforced based on roles:

1. Authentication Endpoint (/api/auth/**): Publicly accessible. Users can register and log in to receive a JWT access token.
2. ADMIN Role: Can manage patient and doctor records, view overall hospital statistics, and delete entities (/api/admin/**).
3. DOCTOR Role: Can view assigned patient appointments, write medical notes/prescriptions (/api/doctors/**).
4. PATIENT Role: Can view available doctors and book appointments (/api/patients/**).

All requests to protected resources must include the JWT token in the HTTP Header: Authorization: Bearer <JWT_TOKEN>

3. Prerequisites

- MongoDB
- Visual Studio Code
- Java
- Git
- Postman

4. API Specification

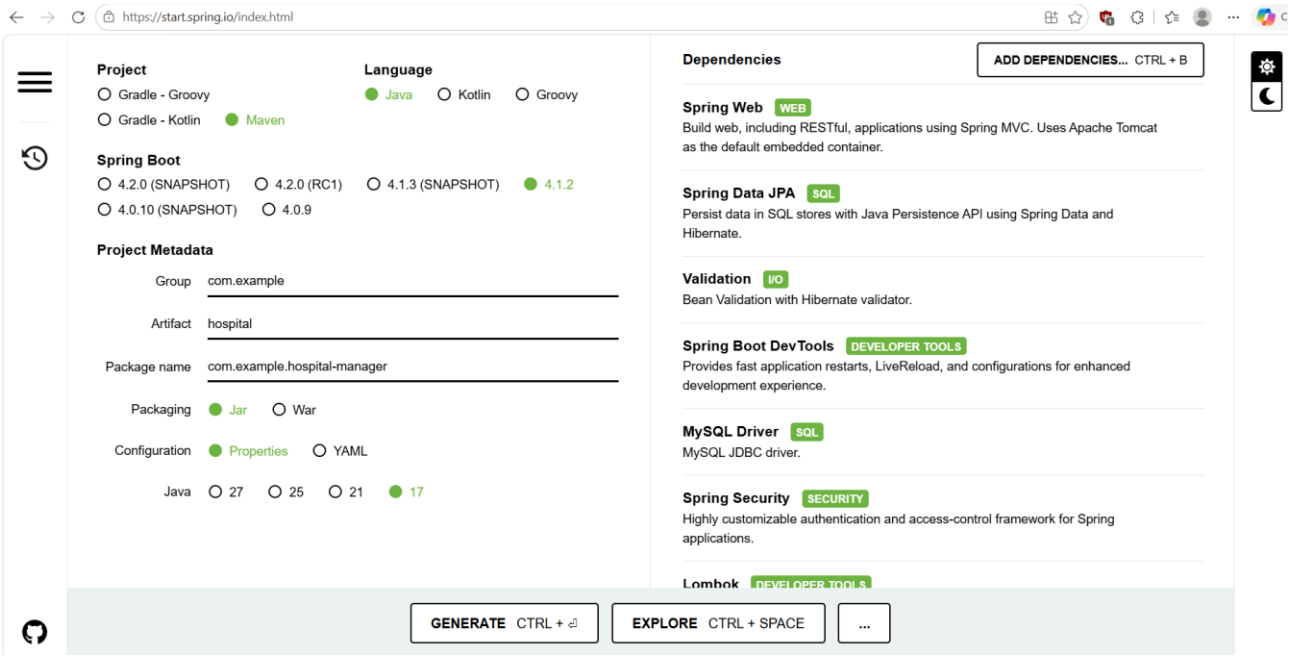
Method	Endpoint	Allowed Roles	Description
POST	/api/auth/register	Public	Register a new user (ADMIN, DOCTOR, PATIENT)
POST	/api/auth/login	Public	Authenticate user & return JWT Token
GET	/api/admin/patients	ROLE_ADMIN	List all registered patients
POST	/api/admin/doctors	ROLE_ADMIN	Add a new doctor profile
GET	/api/doctors/appointments	ROLE_DOCTOR, ROLE_ADMIN	View doctor appointments
POST	/api/patients/appointments	ROLE_PATIENT	Book a doctor appointment

.

5. Tasks

Task 1: Create the Project with Spring Initializr

Open https://start.spring.io in your browser.



Fill in the Project Metadata exactly as shown:

Setting	Value
Project	Maven
Language	Java
Spring Boot	The latest stable release
Group	com.example
Artifact	hospital-manager
Name	hospital-manager
Description	Hospital Management System
Package name	com.example.hospital-manager
Packaging	Jar
Java	8/17/21

3. Click **ADD DEPENDENCIES** (or press Ctrl + B), search for each dependency below and add it. You must end up with **six** dependencies:

Dependency (search name)	**Starter added to pom.xml**
Spring Web	spring-boot-starter-web (-webmvc in Boot 4)
Spring Data JPA	spring-boot-starter-data-jpa
MySQL Driver	MySQL Driver mysql-connector-j
Lombok	lombok
Validation	spring-boot-starter-validation
Spring Boot DevTools	spring-boot-devtools
Spring Security	spring-boot-starter-security
JJWT API (Manual addition)	io.jsonwebtoken:jjwt-api:0.11.5
JJWT Impl (Manual addition)	io.jsonwebtoken:jjwt-impl:0.11.5
JJWT Jackson (Manual addition)	io.jsonwebtoken:jjwt-jackson:0.11.5

Snippet for Manual JJWT Dependencies (pom.xml)

```
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-api</artifactId>
    <version>0.11.5</version>
</dependency>
```

```
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-impl</artifactId>
    <version>0.11.5</version>
    <scope>runtime</scope>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-jackson</artifactId>
    <version>0.11.5</version>
    <scope>runtime</scope>
</dependency>
```

5. You need to Add those dependencies into pom.xml manually.
6. As per the table below you need to create those java classes inside the packages.

Package	Class	Type	Purpose
model	Role	Enum	ADMIN, DOCTOR, NURSE, RECEPTIONIST
	AppointmentStatus	Enum	SCHEDULED, COMPLETED, CANCELLED
	AppUser	Entity	Staff account: username, BCrypt password hash, role
	Patient	Entity	Patient details, with validation rules
	Appointment	Entity	Links a patient and a doctor at a date and time
	MedicalRecord	Entity	Diagnosis and prescription written by a doctor
repository	UserRepository	Repository	Find user by username, list doctors
	PatientRepository	Repository	Search patients by name
	AppointmentRepository	Repository	List appointments (all or per doctor), check if a patient has any
	MedicalRecordRepository	Repository	Records of one patient, check if a patient has any
dto	LoginRequest	Record	Login body: username and password
	RegisterRequest	Record	New staff account body, with validation
	AuthResponse	Record	Login result: token, type, expiry, role
	UserResponse	Record	A user shown to clients, with no password
	AppointmentRequest	Record	Body for booking an appointment (@Future date)

	StatusRequest	Record	Body for changing an appointment status
	AppointmentResponse	Record	Appointment shown to clients
	MedicalRecordRequest	Record	Body for writing a medical record
	MedicalRecordResponse	Record	Medical record shown to clients
	ErrorResponse	Record	JSON shape of every error
exception	ApiException	Exception	Carries an HTTP status (404, 409 ...)
	GlobalExceptionHandler	@RestControllerAdvice	Turns exceptions into JSON 400, 401, 403, 404, 409
security	JwtService **(core)**	Service	Creates and validates JWTs (signature and expiry)
	JwtAuthenticationFilter **(core)**	Filter	Reads Authorization: Bearer ... on every request and logs the user in
	SecurityConfig **(core)**	Configuration	Public URLs, stateless sessions, filter order, BCrypt, @PreAuthorize
	CustomUserDetailsService **(core)**	Service	Loads the user, hash and role from MySQL
	JsonAuthenticationEntryPoint **(core)**	Component	Sends the 401 JSON response
	JsonAccessDeniedHandler	Component	Sends the 403 JSON response
	ErrorResponseWriter	Helper	Writes JSON from inside the filter chain
service	AuthService	Service	Login, register, “me”, list users and doctors
	PatientService	Service	Patient CRUD, with a block on deleting a patient who has data
	AppointmentService	Service	Booking, doctor-only list, ownership rule
	MedicalRecordService	Service	Writes records using the doctor from the token
controller	AuthController	REST controller	/api/auth/login, /register, /me
	StaffController	REST controller	/api/users, /api/doctors
	PatientController	REST controller	/api/patients with role checks
	AppointmentController	REST controller	/api/appointments with role checks
	MedicalRecordController	REST controller	/api/records with role checks
config	DataLoader	CommandLineRunner	Creates the seeded accounts and sample patients on the first start

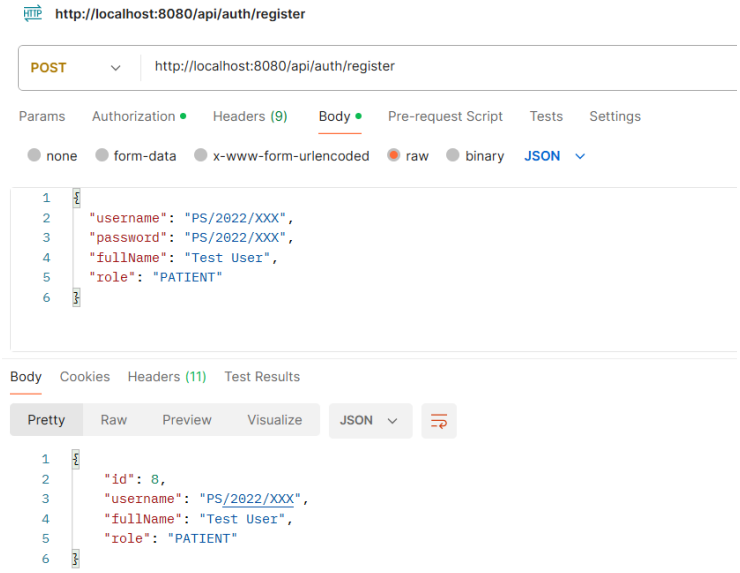
Task 2: Modify the Administrator's Role by Tampering with the JWT Token

Question: A Patient logs into the system and receives a JWT token. Modify the role claim inside the JWT payload from PATIENT to ADMIN using a JWT Decoder or Base64 Encoder. Then, use the modified token to attempt to register a new nurse. Does the modified token work?

Ex-:

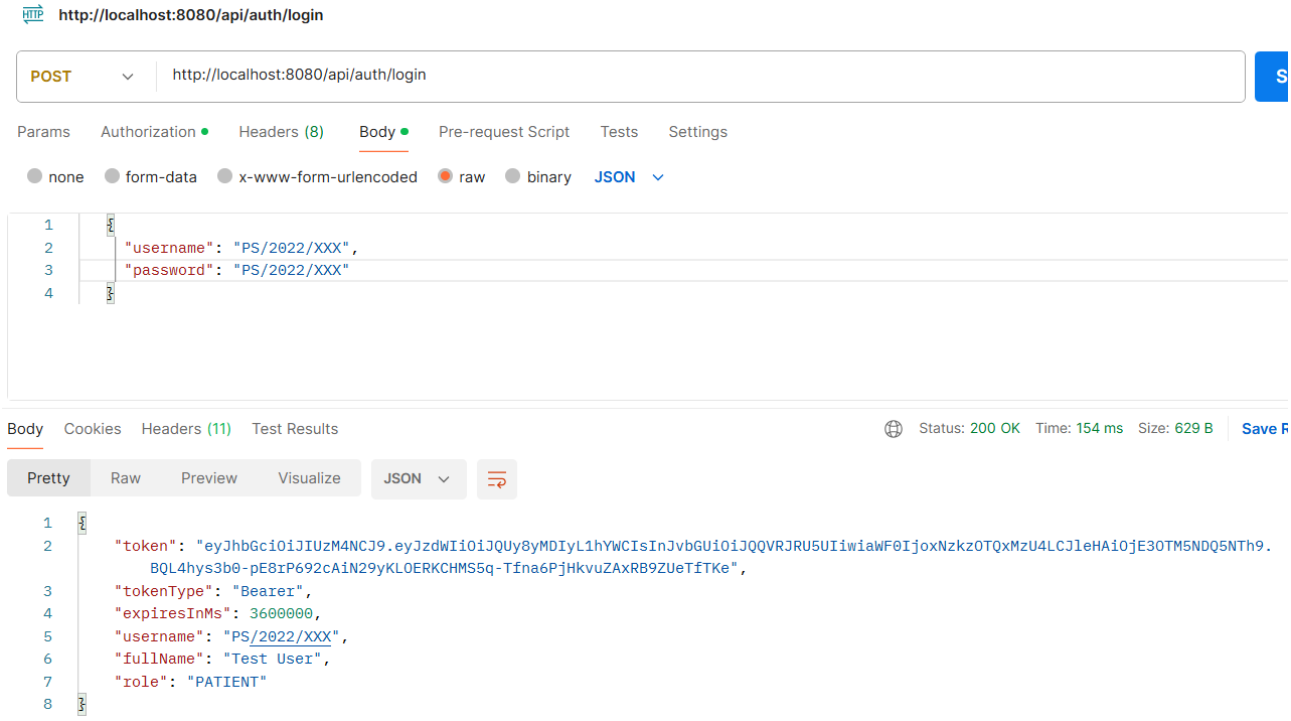
Step 1: Log in as an Administrator

1. Open Postman.
2. Add new Patient (Username and Password must be your Student Number: XX/2022/XXX)



3. Send a login request using the Patient credentials.
   (Method: POST, URL: http://localhost:8080/api/auth/login)
4. Copy the generated JWT token from the response.

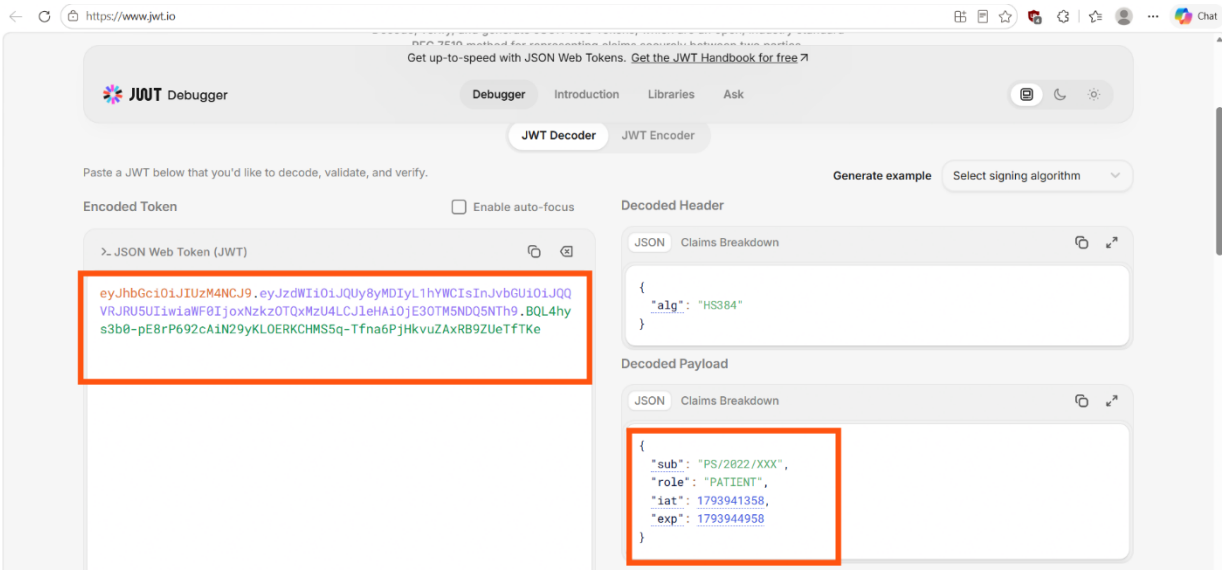
Screenshot 01: Capture the successful login response showing the generated JWT token.



Step 2: Decode the JWT Token

1. Open a JWT Decoder (e.g., JWT.io).
2. Paste the JWT token obtained in Step 1.
3. Observe the three JWT components:
    - Header
    - Payload
    - Signature
5. Examine the payload and identify the role claim.
6. Confirm that the current role is PATIENT.

Ex-:



Step 3: Modify the JWT Payload

1. Update the patient role from PATIENT to DOCTOR in payload.

Ex-:

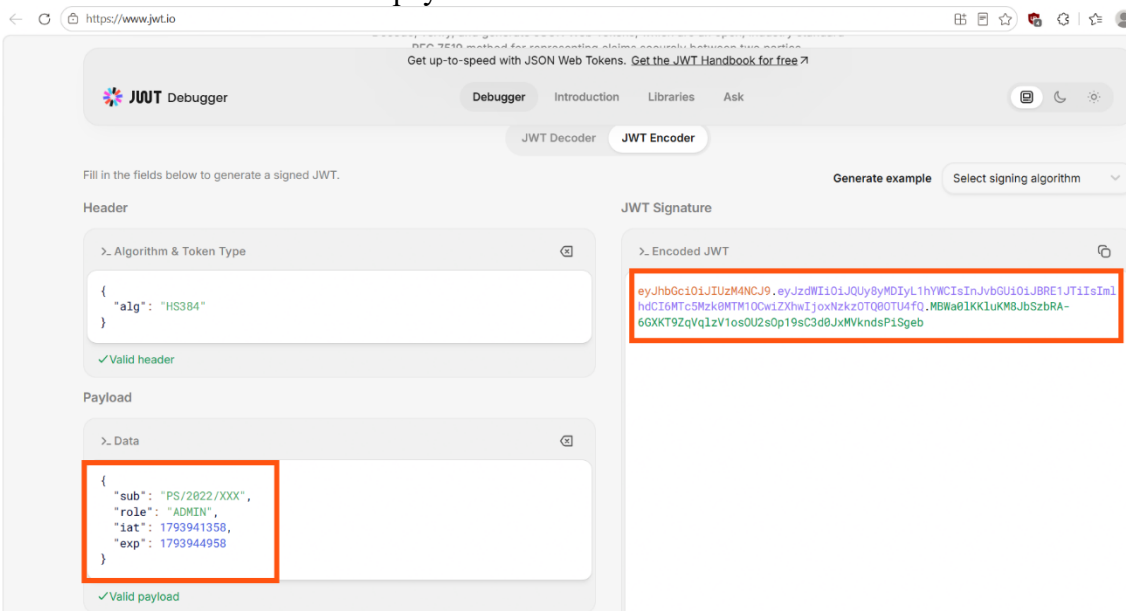
Original payload:

```
{
  "sub": "PS/2022/XXX",
  "role": "PATIENT",
  "iat": 1793941358,
  "exp": 1793944958
}
```

Modified payload:

```
{
  "sub": "PS/2022/XXX",
  "role": "ADMIN",
  "iat": 1793941358,
  "exp": 1793944958
}
```

2. Encode the modified JSON payload.



Step 4: Use the Encoded JWT token to add new Nurse

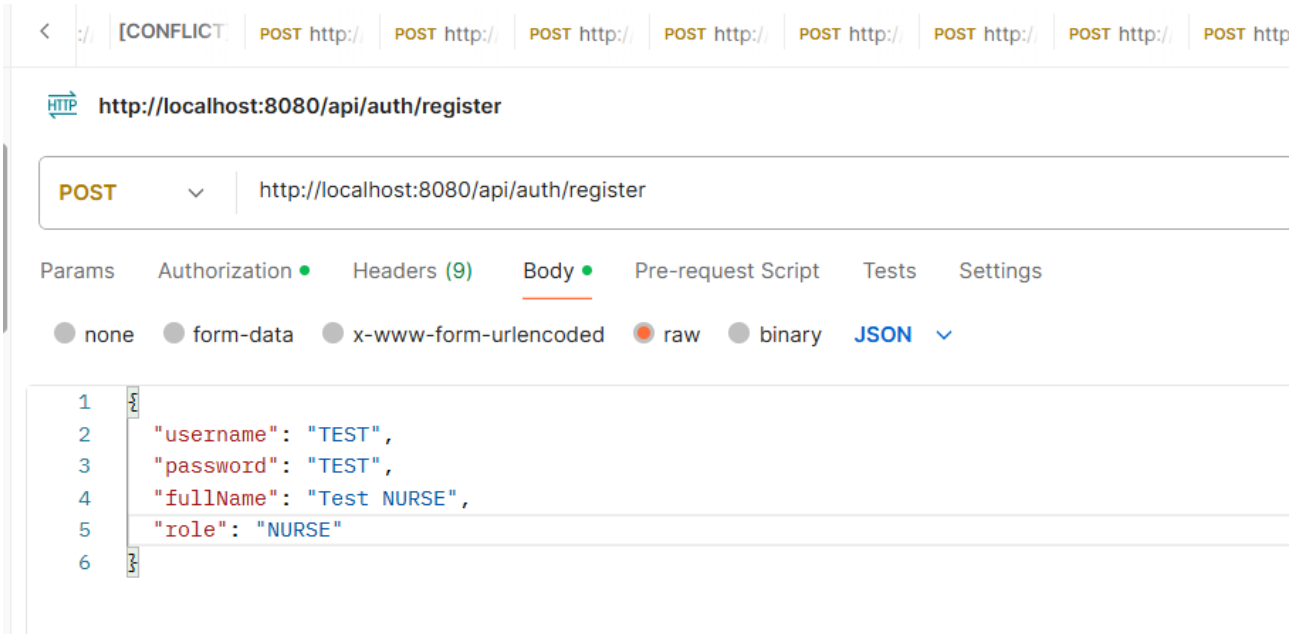
1. Return to Postman.

1. Create a new request to add a nurse (Username and Password must be your Student Number: XX/2022/XXX).

2. Select the HTTP method POST.

3. Enter the nurse registration API endpoint.

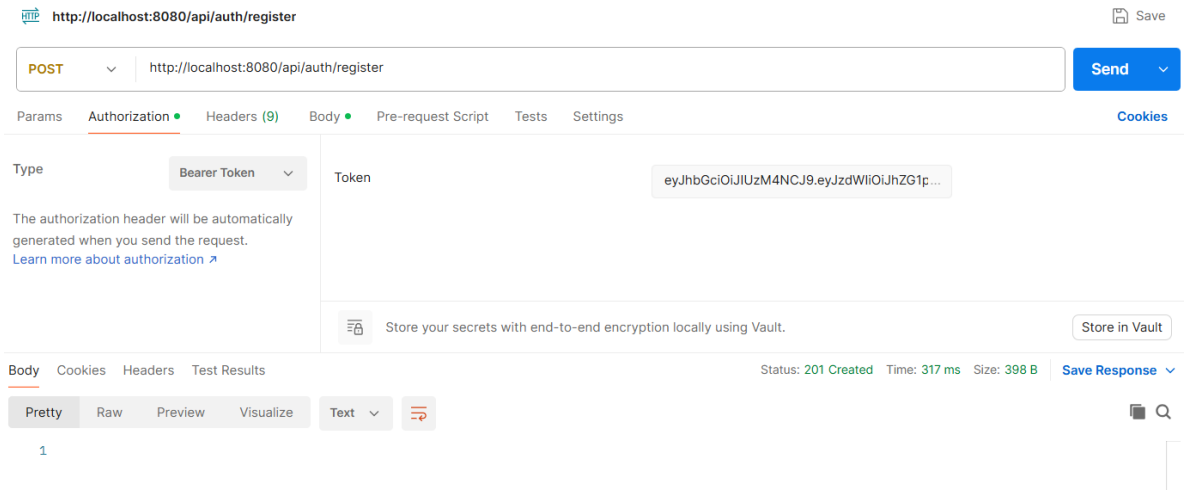
Ex-:



4. Navigate to the Authorization tab.

5. Select Bearer Token.

6. Paste the JWT token copied in Step 1. Do not generate a new token.



7. Navigate to Body → raw → JSON.
8. Enter the following example request:

```
{
   "name": "Test",
   "username": "Your Std No",
   "password": " Your Std No",
   "role": "NURSE"
}
```

9. Click Send.
10. Observe the Response (Check the response returned by the server.)

Step 5: Use the same token to view the patient records

1. Return to Postman.
2. Create a new request to view the patient records. (GET: http://localhost:8080/api/patients)

Submission Requirements

- Submit as a PDF including all the screenshots and code. Include your name, student ID, date, course code, course name.
- Deadline: 20th of November 2026